МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 1

Тема: Порождающие паттерны: Одиночка, Строитель

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки
09.03.04 «Программная инженерия»

Постников Илья Викторович

Проверил:

Доцент ДЦРСИЭ

Альбекова З. М.

Ставрополь 2026

Цель: изучить принцип работы порождающего паттерна проектирования «Одиночка» (Singleton) и реализовать прикладную задачу «Графический пароль» с использованием данного паттерна.

Задание: необходимо разработать приложение для работы с графическим паролем и реализовать один из порождающих паттернов проектирования - «Строитель» или «Одиночка».

В соответствии с вариантом задания был выбран паттерн **«Одиночка» (Singleton)**.

Вариант 8 - Графический пароль.

Разработанное приложение позволяет создавать, сохранять и проверять графический пароль, заданный последовательностью выбранных ячеек.

1. Описание паттерна «Одиночка»

Паттерн «Одиночка» (Singleton) - это порождающий паттерн проектирования, который гарантирует, что для определённого класса будет создан только один экземпляр, и предоставляет единую точку доступа к этому экземпляру.

Данный паттерн применяется в ситуациях, когда наличие нескольких экземпляров одного класса может быть нежелательным или не имеет смысла.

В разработанном приложении паттерн Singleton используется в классе PasswordManager. Данный класс отвечает за создание, сохранение, проверку и удаление графического пароля.

Для реализации паттерна в классе используется переменная _instance и переопределённый метод __new__(). При создании объекта проверяется наличие уже существующего экземпляра класса. Если экземпляр отсутствует, он создаётся. При последующих обращениях возвращается уже существующий объект.

Таким образом, в приложении используется единый экземпляр PasswordManager.

2. Описание разработанного приложения

Разработанное приложение предназначено для создания и проверки графического пароля.

Приложение реализовано на языке **Python** с использованием библиотеки **Tkinter** для создания графического интерфейса пользователя.

Основное окно приложения содержит сетку из девяти ячеек, пронумерованных от 1 до 9. Пользователь формирует пароль путём последовательного выбора ячеек.

К паролю предъявляются следующие требования:

- длина последовательности составляет от 4 до 9 ячеек;
- одна и та же ячейка не может быть выбрана повторно;
- каждая выбранная ячейка отображается в текущей последовательности;
- созданный пароль сохраняется для последующей проверки.

Приложение содержит два основных режима:

1. **Создание пароля** - пользователь выбирает последовательность ячеек и сохраняет её.
2. **Вход** - пользователь вводит ранее установленную последовательность и выполняет проверку.

Также реализованы кнопка «Очистить» для сброса текущей введённой последовательности, и кнопка «Сбросить сохранённый пароль» для удаления пароля из файла.

3. Хранение и проверка пароля

Для хранения пароля используется файл graphical_password.json.

Сам графический пароль не сохраняется в открытом виде. Перед сохранением последовательность ячеек преобразуется в хэш с использованием алгоритма **SHA-256**.

Например, последовательность:

1 → 5 → 9 → 8

Преобразуется в строку "1598" (конкатенация цифр) и затем хэшируется.

При последующей проверке введённая пользователем последовательность также преобразуется в SHA-256 хэш. Полученное значение сравнивается с сохранённым.

Если значения совпадают, пароль считается правильным. В противном случае программа сообщает о неверном пароле.

4. Диаграмма классов

Для отображения структуры разработанного приложения была создана UML-диаграмма классов.

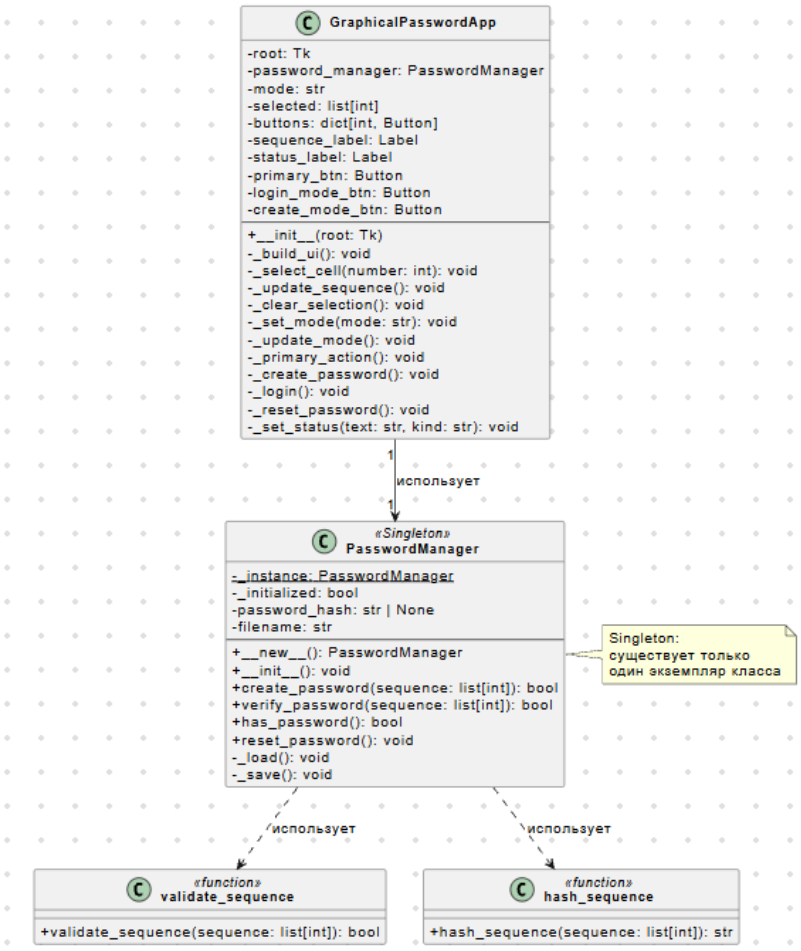


Рисунок 1 - Диаграмма классов приложения «Графический пароль»

Диаграмма содержит два основных класса (GraphicalPasswordApp и PasswordManager) и две вспомогательные функции (validate_sequence и hash_sequence). На диаграмме также отражены атрибуты классов, включая

служебные поля (_initialized, filename), и указана кратность связи между GraphicalPasswordApp и PasswordManager (1:1).

Класс GraphicalPasswordApp отвечает за создание графического интерфейса, обработку действий пользователя, выбор ячеек и взаимодействие с менеджером паролей.

Класс PasswordManager реализует паттерн «Одиночка» и отвечает за операции с графическим паролем.

Связь между классами показывает, что GraphicalPasswordApp использует единственный экземпляр PasswordManager.

5. Реализация паттерна «Одиночка»

Основная часть реализации паттерна находится в классе PasswordManager.

Метод __new__() контролирует создание экземпляра класса:

```
class PasswordManager:

    _instance = None

    def __new__(cls):

        if cls._instance is None:

            cls._instance = super().__new__(cls)

            cls._instance._initialized = False

        return cls._instance



    def __init__(self):

        if self._initialized:

            return
```

```
self._initialized = True

self.password_hash = None

self.filename = "graphical_password.json"

self._load()
```

Флаг _initialized предотвращает повторную инициализацию полей при повторных вызовах PasswordManager(). Метод __init__ загружает сохранённый хэш из файла graphical_password.json при первом создании экземпляра.

При первом создании объекта PasswordManager значение _instance отсутствует, поэтому создаётся новый объект.

При последующих попытках создать PasswordManager новый объект не создаётся. Метод __new__() возвращает уже существующий экземпляр.

Таким образом, класс PasswordManager реализует принцип Singleton.

6. Результаты работы программы

После запуска приложения пользователю отображается главное окно с сеткой из девяти ячеек.

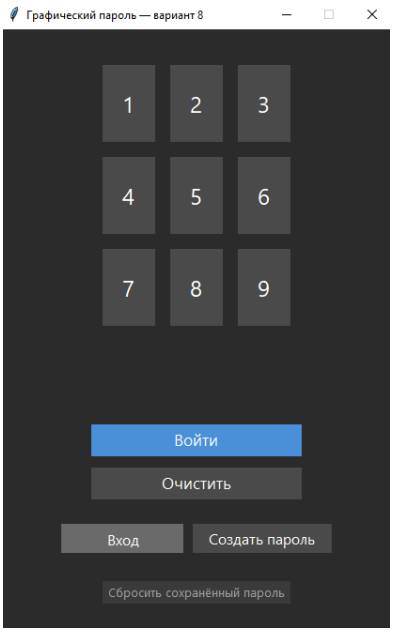


Рисунок 2 - Главное окно приложения

Для создания пароля пользователь переходит в режим «Создать пароль», выбирает необходимую последовательность ячеек и нажимает кнопку «Сохранить пароль». Если пароль уже существует, программа запрашивает подтверждение на его перезапись.

Например:

1 → 5 → 9 → 8

После успешного сохранения программа выводит сообщение об успешном создании пароля.

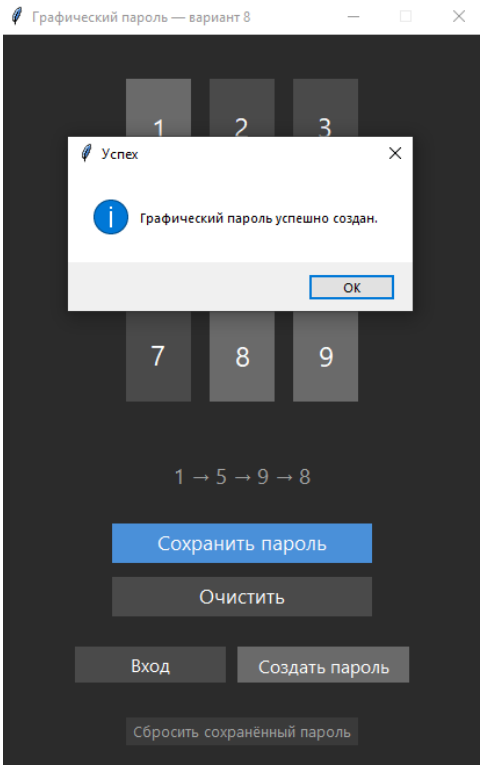


Рисунок 3 - Создание графического пароля

После создания пароля пользователь может перейти в режим входа и повторно ввести сохранённую последовательность.

При правильном вводе программа выводит сообщение:

«Пароль верный. Доступ разрешён.»

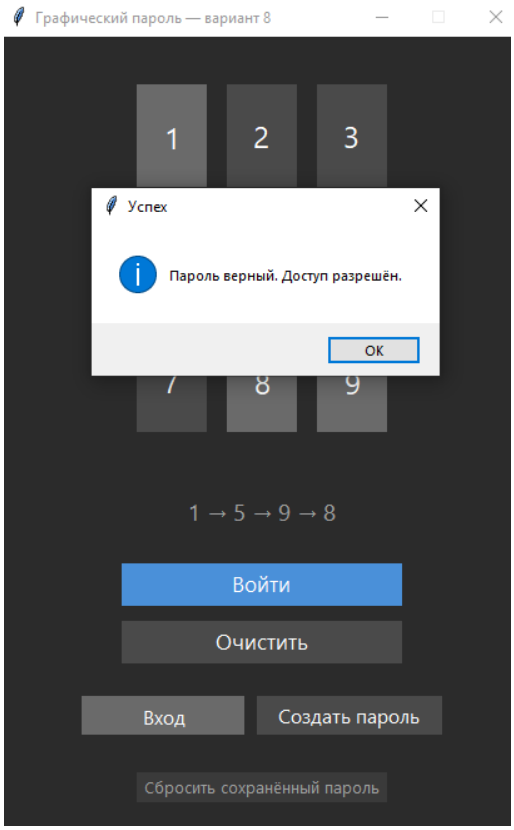


Рисунок 4 - Успешная проверка графического пароля

Если пользователь вводит другую последовательность, программа выполняет сравнение хэшей и выводит сообщение:

«Пароль неверный. Доступ запрещён.»

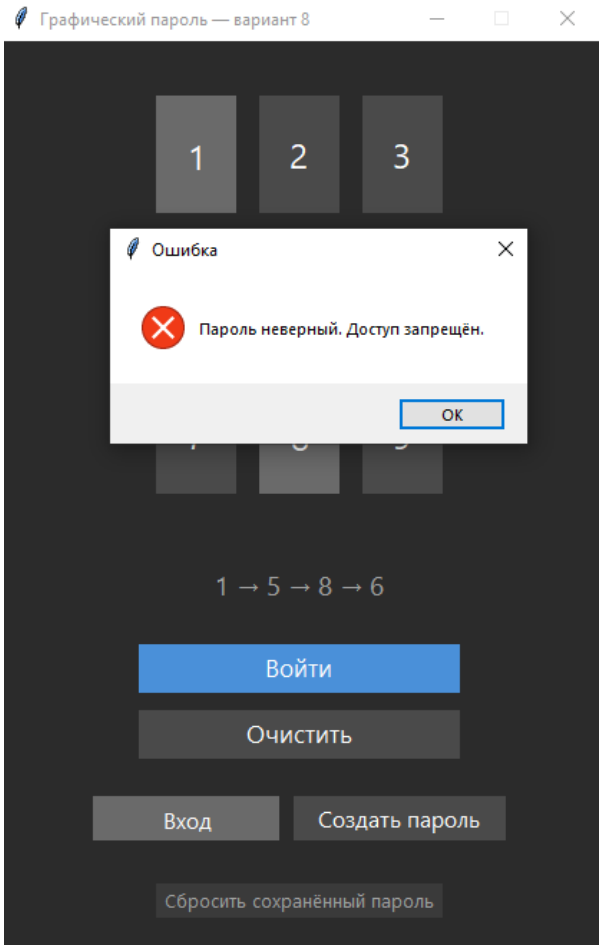


Рисунок 5 - Проверка неправильного графического пароля

Также в приложении предусмотрена возможность удаления сохранённого пароля. При нажатии на кнопку «Сбросить сохранённый пароль» появляется диалоговое окно подтверждения с кнопками «Да» и «Нет».

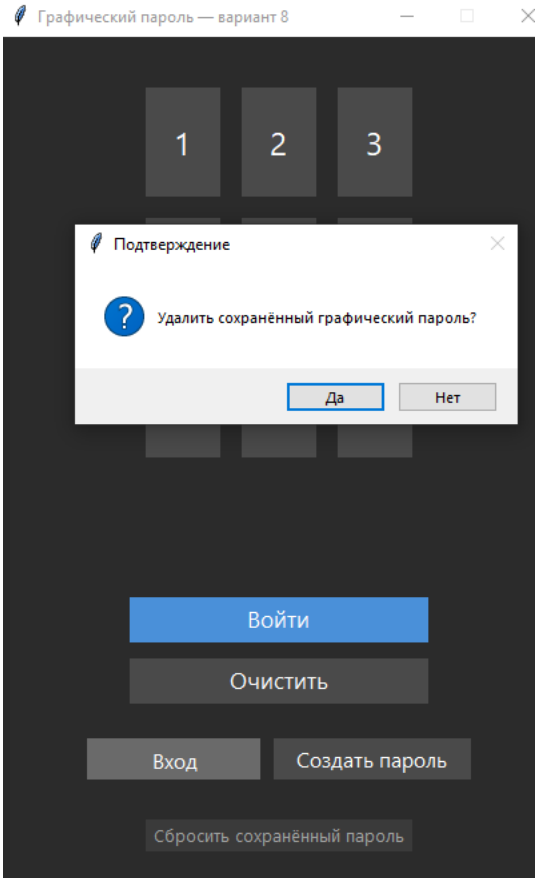


Рисунок 6 - Сброс сохранённого пароля

7. Исходный код

Приложение разработано на языке программирования Python. Для создания графического интерфейса используется библиотека Tkinter.

Основные элементы программы:

- GraphicalPasswordApp - класс графического интерфейса;
- PasswordManager - класс управления паролем и реализации Singleton;
- validate_sequence() - вспомогательная функция проверки корректности последовательности;
- hash_sequence() - вспомогательная функция получения SHA-256 хэша;
- graphical_password.json - файл хранения хэша пароля.

Полный исходный код программы представлен в приложении к лабораторной работе. Код доступен в репозитории GitHub:

https://github.com/q1wf3/Patterny_Postnikov

Вывод: в ходе выполнения лабораторной работы было разработано приложение «Графический пароль» на языке программирования Python.

В приложении реализованы создание, сохранение, проверка и удаление графического пароля. Для хранения пароля используется файл JSON, а для защиты сохранённых данных применяется хэширование SHA-256.

В процессе разработки был изучен и реализован порождающий паттерн проектирования **«Одиночка» (Singleton)**. Паттерн применён к классу PasswordManager, благодаря чему в приложении используется единственный экземпляр менеджера паролей.

Таким образом, была изучена практическая реализация паттерна Singleton и получено работоспособное приложение с графическим интерфейсом для создания и проверки графического пароля. Приложение также реализует валидацию вводимой последовательности (длина от 4 до 9, без повторений) и защищённое хранение пароля в виде SHA-256 хэша.